Guía de estudio — Demo del 16/10 (Sistema de Alerta Temprana, Grupo 6)

Oct 13, 2026 · @Gianluca

1. Qué es el sistema

El sistema responde una sola pregunta: ¿a quién llamo esta semana? Calcula un semáforo de riesgo de abandono para cada alumno con dos señales, asistencia y entregas, y los muestra ordenados por prioridad.

El problema de la clienta. Una tutora sigue dos comisiones de primer año, unos sesenta alumnos. La asistencia está en un cuaderno y las entregas en el aula virtual. Cruzarlas a mano lleva una tarde, así que no lo hace y se entera tarde de quién dejó de venir. Hay un solo usuario: quien sigue la comisión, sin login.

El recorrido de punta a punta

1. Se cargan los alumnos de una comisión, a mano o por CSV.
2. Se registra la asistencia de cada clase.
3. Se crean los trabajos prácticos y se marca quién entregó cada uno.
4. El sistema calcula el riesgo de cada alumno: rojo, amarillo o verde.
5. El panel lista a los alumnos de una comisión de mayor a menor riesgo, con el dato que justifica el color.
6. Desde el panel se abre la ficha de un alumno con su historial.

Por qué dos señales. La clienta dice que la señal principal es la asistencia, pero todos los casos que cuenta empezaron por dejar de entregar. Joaquín, de la comisión A en los datos de demo, viene a todas las clases y entregó 1 de 3 trabajos: con solo asistencia saldría verde, con las dos señales sale rojo.

Cómo se relaciona con la consigna. El centro del proyecto es el indicador, no el registro de datos. Cargar alumnos y asistencia es transaccional; lo que habilita una decisión es el cálculo de riesgo y el panel ordenado.

2. Qué desarrollamos y qué no

Para el 16/10 hay 7 historias desarrolladas contra las 6 que exige el calendario, y las 11 están escritas en Trello. "Creación" es escribir la historia; "desarrollo" es programarla.

Historia	Qué pide	Estado	Dónde está el código
H1 Nómina	Cargar y ver los alumnos de una comisión	Hecha	`EstudianteViewModel.cs`, `EstudiantePage.xaml`, `CargaMasivaViewModel.cs`, `CargaMasivaPage.xaml`, `CsvImporterService.cs`, `ValidacionEmail.cs`
H2 Calendario	Definir y consultar clases y trabajos	Pendiente (parcial)	Las clases nacen al guardar asistencia y los trabajos se crean en Entregas
H3 Registrar asistencia	Marcar presentes de una fecha	Hecha	`AsistenciaFechaViewModel.cs`, `AsistenciaFechaPage.xaml`
H4 Corregir asistencia	Cambiar una fecha ya registrada	Hecha	La misma pantalla que H3
H5 Entregas	Registrar quién entregó cada trabajo	Hecha	`EntregasViewModel.cs`, `EntregasPage.xaml`
H6 Riesgo	Calcular el riesgo con asistencia y entregas	Hecha	`RiesgoService.cs`, `RiesgoServiceTests.cs`
H7 Umbrales	Configurarlos sin tocar el código	Pendiente	Hoy están fijos en `RiesgoService.cs`
H8 Panel	Comisión ordenada por riesgo, con el detalle	Hecha	`PanelRiesgoViewModel.cs`, `PanelRiesgoPage.xaml`
H9 Ficha	Historial de un estudiante	Hecha	`DetalleEstudianteViewModel.cs`, `DetalleEstudiantePage.xaml`, `SeleccionAlumnoService.cs`
H10 Contacto	Registrar un contacto y su resultado	Pendiente	Todavía sin código
H11 Intervenciones	Mostrarlas en el panel	Pendiente	Todavía sin código

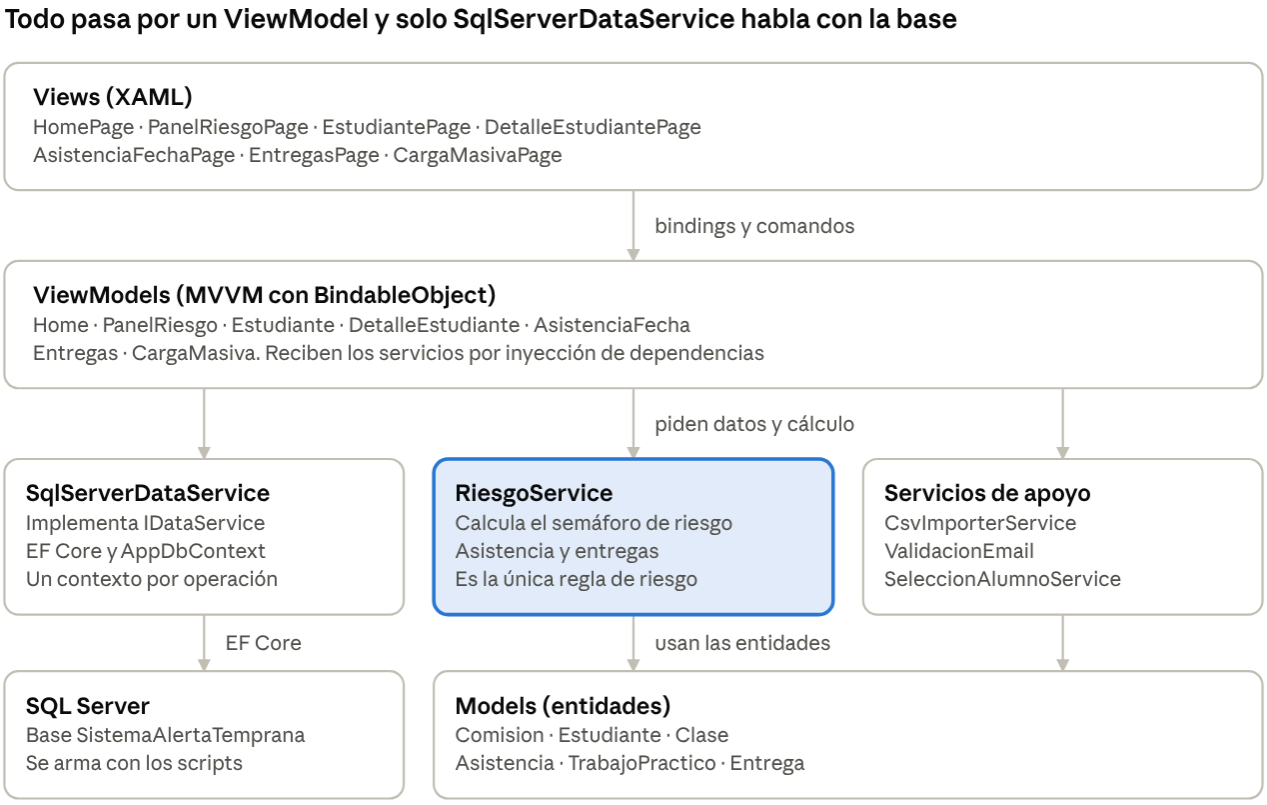
H2, H7, H10 y H11 están planificadas para los sprints 4 a 7 del calendario. Su propio análisis de requerimientos marca los umbrales configurables como deseables pero postergables.

Estructura de carpetas (dentro de `ProyectoFinal-PPII-G6/`)

- `Models/`: las entidades de la base (`Comision`, `Estudiante`, `Clase`, `Asistencia`, `TrabajoPractico`, `Entrega`).
- `ViewModels/`: la lógica de cada pantalla.
- `Views/`: las pantallas en XAML. La carpeta se llama `Views` con V mayúscula y el namespace también.
- `Services/`: acceso a datos (`IDataService`, `SqlServerDataService`, `AppDbContext`), cálculo (`RiesgoService`) y utilidades.
- `MauiProgram.cs` y `AppShell.xaml`: registro de piezas y navegación.
- Fuera de la app: `ProyectoFinal-PPII-G6.Tests/`, `scripts/` y `datos_demo/`.

3. Arquitectura y cómo se conectan las piezas

La app tiene cuatro capas y una regla simple: cada pantalla habla solo con su ViewModel, y solo `SqlServerDataService` habla con la base.



arquitectura · 4 capas, los servicios y la base

Las flechas muestran quién le pide qué a quién. `RiesgoService`, resaltado, es donde vive la regla del semáforo.

Cómo viaja un dato: abrir el panel de riesgo

1. `PanelRiesgoPage.OnAppearing` llama a `PanelRiesgoViewModel.CargarAsync`.
2. El ViewModel le pide a `IDataService` los alumnos, clases, trabajos, asistencias y entregas de la comisión elegida.
3. `SqlServerDataService` abre un `AppDbContext`, consulta SQL Server con Entity Framework Core y devuelve entidades.
4. El ViewModel le pasa a `RiesgoService` las asistencias y entregas de cada alumno y los totales de su comisión.
5. `RiesgoService` devuelve los porcentajes y el nivel; el ViewModel arma una fila por alumno y las ordena.
6. La página muestra la lista porque está enlazada (binding) a la colección `Alumnos`.

Inyección de dependencias. `MauiProgram.cs` registra cada pieza una sola vez: los servicios como singleton (una sola instancia) y los ViewModels y las páginas como transient (una nueva cada vez que se piden). Una página pide su ViewModel por constructor y un ViewModel pide los servicios que usa; nadie crea lo que necesita con `new`.

Navegación. `AppShell.xaml` define las pestañas y sus rutas, y `Shell.Current.GoToAsync` cambia de una a otra. Para abrir la ficha de un alumno desde el panel, `SeleccionAlumnoService` guarda qué alumno hay que mostrar en el historial.

Qué cambia si cambia la base. Solo hay que escribir otra implementación de `IDataService`. Ningún ViewModel ni pantalla se toca.

4. Base de datos y scripts

La base es SQL Server, se arma con scripts y la app la usa a través de Entity Framework Core. No hay migraciones: lo que define el esquema es `scripts/01_crear_base.sql`.

Tabla	Columnas clave	Restricciones
`Comisiones`	Id, Nombre	Nombre único
`Estudiantes`	Id, DNI, Nombre, Apellido, Email, ComisionId	Clave foránea a la comisión; DNI único dentro de cada comisión
`Clases`	Id, Fecha, ComisionId, Descripcion	Clave foránea a la comisión; una sola clase por comisión y fecha

Tabla	Columnas clave	Restricciones
`TrabajosPracticos`	Id, ComisionId, Titulo, FechaEntrega	Clave foránea a la comisión
`Asistencias`	Id, EstudianteId, ClaseId, Presente, Justificada	Claves foráneas con borrado en cascada; un registro por alumno y clase
`Entregas`	Id, EstudianteId, TrabajoPracticoId, Entregado	Claves foráneas; un registro por alumno y trabajo

Cómo se relacionan. Una comisión tiene muchos alumnos, clases y trabajos. Un alumno tiene muchas asistencias y entregas. Una clase tiene asistencias de muchos alumnos, y un trabajo tiene entregas de muchos alumnos.

Las restricciones son la red de seguridad. La pantalla avisa antes de guardar un DNI repetido, pero si algo se le escapa la base lo rechaza. Lo mismo pasa con dos asistencias del mismo alumno en la misma clase.

Los scripts, en orden

Script	Qué hace	Cuidado
`01_crear_base.sql`	Borra la base si existe y la crea con todas las tablas	Borra todos los datos
`02_usuario_app.sql`	Le da acceso al usuario `app_maui`	Solo hace falta si se usa login de SQL Server; con una base local y autenticación de Windows se saltea
`03_comisiones.sql`	Crea las comisiones A, B y C	Se puede repetir sin duplicar
`04_datos_demo.sql`	Carga 12 alumnos con 6 clases y 3 trabajos por comisión	Las fechas se calculan hacia atrás desde hoy: correrlo el día de la demo o el anterior
`mantenimiento/limpiar_datos.sql`	Vacía los datos, reinicia los ids y deja A, B y C	Hay que correrlo con una cuenta administradora

Cómo se conecta la app. `AppDbContext` toma la conexión de la variable de entorno `PPII_DB`. Si no existe, usa `Server=.\SQLEXPRESS;Database=SistemaAlertaTemprana;Trusted_Connection=True;TrustServerCertificate=True;`, es decir, SQL Server Express local con tu usuario de Windows. La clave de ningún servidor está en el repositorio.

Un contexto por operación. `SqlServerDataService` crea un `AppDbContext` nuevo en cada método y lo descarta al terminar. Por eso la interfaz `IDataService` se registra como singleton sin riesgo de compartir estado entre pantallas.

5. El cálculo de riesgo

El nivel es rojo si la asistencia es menor al 75 % o las entregas menores al 60 %, y amarillo si son menores al 85 % o al 80 %. Todo vive en `RiesgoService.CalcularRiesgo`, el único lugar donde está la regla.

```
\text{asistencia} = \frac{\text{presentes} + 0{,}5 \times \text{justificadas}}{\text{total de clases}} \times 100
```

```
\text{entregas} = \frac{\text{trabajos entregados}}{\text{total de trabajos}} \times 100
```

Nivel	Condición (basta con que se cumpla una)	Texto en pantalla
Rojo	Asistencia < 75 % o entregas < 60 %	ALTO RIESGO
Amarillo	Asistencia < 85 % o entregas < 80 %	RIESGO MODERADO
Verde	Ninguna de las anteriores	AL DÍA

Reglas que conviene saber de memoria

- Una ausencia justificada vale la mitad de una injustificada.
- Una clase sin registro de asistencia cuenta como ausencia.
- Un alumno sin registro de entrega cuenta como no entregado.
- Si la comisión no tiene clases o no tiene trabajos, esa señal vale 100 % y no penaliza.
- Los totales son los de la comisión del alumno, nunca los de otra.
- El riesgo no se guarda en la base: se recalcula cada vez que se abre una pantalla, así nunca queda desactualizado.

Ejemplo con una justificada. Un alumno con 5 presentes y 1 ausencia en 6 clases tiene 83,3 % si la falta es injustificada (amarillo) y 91,7 % si es justificada (verde).

Qué tiene que mostrar el panel con los datos de demo. El orden es por nivel, y dentro de cada nivel va primero el peor de los dos porcentajes.

Comisión	Alumno	Asistencia	Entregas	Nivel
A	Joaquín Pérez	6/6 (100 %)	1/3 (33 %)	Rojo
A	Valentina Díaz	4/6 (67 %)	3/3 (100 %)	Rojo
A	Mateo Gómez	5/6 (83 %)	3/3 (100 %)	Amarillo
A	Lucía Fernández	6/6 (100 %)	3/3 (100 %)	Verde
A	Sofía Benítez	6/6 (100 %)	3/3 (100 %)	Verde
B	Santiago Ruiz	3/6 (50 %)	0/3 (0 %)	Rojo
B	Camila Torres	6/6 (100 %)	2/3 (67 %)	Amarillo
B	Martina López	6/6 (100 %)	3/3 (100 %)	Verde
B	Tomás Silva	6/6 (100 %)	3/3 (100 %)	Verde
C	Facundo Medina	4/6 (67 %)	2/3 (67 %)	Rojo
C	Julieta Herrera	5/6 (83 %)	2/3 (67 %)	Amarillo
C	Nicolás Acosta	6/6 (100 %)	3/3 (100 %)	Verde

Los casos para contar: Joaquín (viene siempre pero dejó de entregar), Santiago (el más grave) y Julieta (amarilla por las dos señales a la vez).

6. Recorrido por cada pantalla

Cada pantalla tiene una página XAML, un ViewModel y una decisión que habilita. Los títulos de las pestañas pueden variar en tu barra (por ejemplo "Panel de riesgo", "Carga de alumnos", "Historial de asistencia", "Registro de asistencia"); acá uso el nombre del archivo.

Inicio (`HomePage` y `HomeViewModel`)

- **Qué hace:** muestra el resumen general, con el total de alumnos y cuántos están en riesgo alto, y permite cambiar entre tema claro y oscuro.
- **Decisión que habilita:** ver de un vistazo si hay un problema grande antes de entrar al detalle.
- **Por dentro:** `CargarMetricasAsync` recalcula el riesgo de todos los alumnos con `RiesgoService`, usando las clases y trabajos de la comisión de cada uno.

Panel de riesgo (`PanelRiesgoPage` y `PanelRiesgoViewModel`)

- **Qué hace:** con una comisión elegida, lista a sus alumnos de mayor a menor riesgo, cada uno con el dato que justifica el color. Tocar un alumno abre su ficha.
- **Decisión que habilita:** a quién contactar primero esta semana.
- **Por dentro:** `CargarAsync` carga las comisiones y conserva la elegida. `CargarAlumnosAsync` trae alumnos, clases, trabajos, asistencias y entregas de esa comisión, llama a `RiesgoService` y arma una fila (`AlumnoRiesgoItem`) con un orden (0 rojo, 1 amarillo, 2 verde, 3 sin datos) y el peor porcentaje para desempatar.
- **Al tocar un alumno:** `SeleccionAlumnoService.Pedir(id)` anota cuál abrir y `Shell.Current.GoToAsync("//HistorialEstudiantes")` cambia de pestaña.

Historial del alumno (`DetalleEstudiantePage` y `DetalleEstudianteViewModel`)

- **Qué hace:** se elige comisión y alumno, y se ve su ficha: datos, semáforo, el detalle numérico, cada clase con fecha, tema y estado, y cada trabajo con su estado.
- **Decisión que habilita:** entender por qué está en riesgo y qué decirle al llamarlo.
- **Por dentro:** `CargarListaEstudiantesAsync` toma del servicio el alumno pedido desde el panel, si hay uno. `CargarDetalleAsync` calcula el riesgo con los totales de la comisión del alumno. Una bandera interna (`_actualizando`) evita que los dos selectores se disparen cargas entre sí.

Alumnos (`EstudiantePage` y `EstudianteViewModel`)

- **Qué hace:** un selector de comisión gobierna el alta y la nómina. Al elegir una comisión aparece su lista, y el alumno nuevo se carga en ella.
- **Decisión que habilita:** quién integra cada comisión.
- **Por dentro:** valida campos obligatorios, formato de email y DNI repetido antes de ir a la base. Después del alta recarga la nómina y deja la comisión elegida para cargar varios seguidos.

Asistencia por fecha (`AsistenciaFechaPage` y `AsistenciaFechaViewModel`)

- **Qué hace:** se elige comisión y fecha. Cada alumno aparece con un interruptor de presente y, si está ausente, otro de justificada. Sirve para tomar la asistencia (H3) y para corregirla (H4).
- **Decisión que habilita:** es la fuente de la señal de asistencia, y permite corregir un error sin tocar la base.
- **Por dentro:** si la fecha no tiene clase, `GuardarAsync` la crea con la comisión elegida. Rechaza fechas futuras. Los alumnos sin registro aparecen como presentes. Después de guardar recarga la lista para que las filas nuevas tomen su id, y así un segundo

guardado actualiza en lugar de duplicar.

Entregas (`EntregasPage` y `EntregasViewModel`)

- **Qué hace:** permite crear un trabajo práctico (título y fecha de entrega) y marcar quién lo entregó.
- **Decisión que habilita:** es la fuente de la segunda señal, la que detecta a quien viene a clase pero dejó de entregar.
- **Por dentro:** no deja crear dos trabajos con el mismo título en una comisión. El valor por defecto es el contrario al de asistencia: sin registro cuenta como no entregado.

Carga masiva (`CargaMasivaPage` y `CargaMasivaViewModel`)

Se explica en la sección 7.

7. Carga masiva por CSV y validaciones

La carga masiva muestra primero una vista previa y solo importa las filas marcadas como nuevas, todas juntas o ninguna. Así un DNI repetido no deja la importación a medias.

Formato del archivo. Una fila por alumno, con encabezado `Dni;Nombre;Apellido;Email`, separado por punto y coma o coma, guardado como CSV UTF-8. El archivo de ejemplo es `datos_demo/alumnos_demo.csv`.

El flujo

1. Elegir la comisión de destino en el selector.
2. Tocar "Elegir archivo CSV". `CsvImporterService.ParseCsvAsync` lee el archivo.
3. La pantalla compara cada fila con los alumnos que ya tiene esa comisión y muestra su estado.
4. Tocar "Importar alumnos". `SaveEstudiantesMassiveAsync` guarda las filas nuevas en un solo paso.

Si se cambia la comisión después de elegir el archivo, la vista previa se evalúa de nuevo contra la comisión nueva.

Estados de la vista previa

Estado	Cuándo aparece	Se importa
Nuevo	Datos completos y DNI que la comisión no tiene	Sí
Ya existe en la comisión	El DNI ya está cargado en esa comisión	No

Estado	Cuándo aparece	Se importa
DNI repetido en el archivo	La misma fila aparece antes en el archivo	No
Datos incompletos	Falta DNI, nombre, apellido o email	No
Email inválido	El email no tiene formato de dirección	No
Dato demasiado largo	DNI, nombre o apellido exceden el largo de la columna	No

Con el archivo de ejemplo y la comisión A, el resumen tiene que decir 6 para importar, 1 ya existe (Lucía), 1 repetido en el archivo y 1 con errores (la fila sin nombre).

Validaciones del alta manual (`EstudianteViewModel.GuardarEstudianteAsync`)

- DNI, nombre, apellido, email y comisión son obligatorios.
- El email se valida con `ValidacionEmail.EsValido`: no vacío, formato de dirección, dominio con punto y hasta 100 caracteres. La misma regla usa la carga masiva.
- El DNI se guarda sin puntos ni espacios, así que `30.123.456` y `30123456` cuentan como el mismo.
- Si el DNI ya existe en la comisión, la pantalla avisa antes de ir a la base. La restricción única de la base queda como respaldo.

Límites que conviene conocer

- Las filas del archivo con menos de tres columnas se ignoran sin avisar.
- Si el archivo no está en UTF-8, los acentos se ven mal.
- El DNI es único por comisión, no en todo el sistema: el mismo DNI puede estar en dos comisiones.

8. Pruebas unitarias

Hay 25 pruebas xUnit en dos archivos: 13 sobre el cálculo de riesgo (H6) y 12 sobre la validación del email (H1). Ninguna toca la base de datos.

Archivo	Pruebas	Qué verifica
`RiesgoServiceTests.cs`	13	El cálculo del semáforo y sus bordes
`ValidacionEmailTests.cs`	12	La regla de validación del email

Las 13 de `RiesgoServiceTests`

- Estudiante cumplidor: verde con 100 % en las dos señales.
- Asistencia en rango medio (80 %): amarillo.
- Asistencia baja (70 %): rojo.
- Sin clases ni trabajos registrados: verde y 100 %.
- Asistencia exactamente 75 %: amarillo, porque el rojo es "menor a 75".
- Asistencia exactamente 85 %: verde, porque el amarillo es "menor a 85".
- Asistencia perfecta con entregas bajas (40 %): rojo.
- Una señal amarilla y otra roja: gana la más grave.
- Las ausencias no suman como presentes.
- Una justificada vale la mitad de una injustificada (91,7 % contra 83,3 %).
- Dos justificadas equivalen a una presente.
- Una justificada puede sacar a un alumno del rojo (de 70 % a 75 %).
- Un registro presente marcado como justificada no suma de más.

Las 12 de `ValidacionEmailTests`: tres emails correctos (uno con espacios alrededor), ocho vacíos o mal formados (nulo, vacío, solo espacios, sin arroba, sin usuario, sin dominio, dominio sin punto, con nombre delante) y uno más largo que la columna.

Cómo se corren. En Visual Studio, menú Test, Run All Tests: tienen que salir 25 aprobadas. Desde la terminal, `dotnet test` en la raíz del repositorio. El proyecto de pruebas referencia al de la app.

Qué no está cubierto. No hay pruebas de integración ni de aceptación todavía: el calendario las ubica en el sprint 8. Las pantallas se verificaron a mano con los datos de demo.

9. Conceptos técnicos para poder explicar el código

En una defensa individual piden explicar con tus palabras qué hace una pieza y por qué está así. Estos son los conceptos que aparecen en el código, cada uno con dónde se ve.

Concepto	En palabras simples	Dónde se ve
MVVM	La pantalla (View) solo muestra; la lógica vive en el ViewModel; los datos son el Model. La View se conecta al ViewModel con bindings	Cada pantalla de `Views/` tiene su ViewModel en `ViewModels/`

Concepto	En palabras simples	Dónde se ve
Binding	Un texto o lista de la pantalla se enlaza a una propiedad del ViewModel y se actualiza solo	`Text="{Binding Estado}"` en los XAML
`BindableObject` y `OnPropertyChanged`	Avisan a la pantalla que una propiedad cambió, para que se redibuje	Los setters de todos los ViewModels
`ObservableCollection`	Una lista que avisa cuando se agregan o quitan elementos	`Filas`, `Alumnos`, `Comisiones`
`Command`	Une un botón con un método del ViewModel	`GuardarCommand`, `ImportarCommand`
`async` y `await`	Esperan a la base de datos sin congelar la pantalla	Todos los métodos que terminan en `Async`
Inyección de dependencias	Una clase no crea lo que necesita: lo pide en el constructor y `MauiProgram.cs` se lo entrega	Los constructores de los ViewModels y las páginas
Singleton y transient	Singleton: una sola instancia para toda la app. Transient: una nueva cada vez que se pide	Servicios singleton, ViewModels y páginas transient
Shell	Define las pestañas y las rutas de navegación	`AppShell.xaml` y `Shell.Current.GoToAsync`
Entity Framework Core	Traduce entre objetos de C# y tablas de SQL Server	`AppDbContext` y `SqlServerDataService`
Restricción única y clave foránea	La base impide duplicados y referencias a filas que no existen	`scripts/01_crear_base.sql`

Concepto	En palabras simples	Dónde se ve
`BindableLayout`	Dibuja una lista de elementos dentro de un layout normal	Las listas del panel, el historial y la vista previa del CSV

Dos patrones propios del código

- **Descartar resultados viejos.** Después de cada consulta, el ViewModel compara la comisión o el alumno que consultó con el que está elegido ahora. Si cambió, ignora el resultado. Evita que una respuesta lenta pise a una más reciente.
- **La bandera `_actualizando`.** Cuando un ViewModel repuebla un selector, el selector cambia solo y dispararía otra carga. La bandera apaga ese disparo mientras se repuebla.

Una frase para explicar por qué usamos interfaces solo en un lugar. Usamos una interfaz para la persistencia (`IDataService`) porque es lo que más puede cambiar. El resto de los servicios son clases concretas, una simplificación válida para un proyecto académico.

10. Decisiones de diseño y su porqué

Cada decisión resuelve un problema concreto que apareció al construir el sistema. Saber cuál era el problema es lo que te permite defenderla.

Decisión	Por qué	Dónde se ve
La comisión es un dato de cada alumno, clase y trabajo	Con tres comisiones, un total de clases global habría dado porcentajes equivocados	Columna `ComisionId` en las tablas; todos los ViewModels filtran por comisión
El riesgo se calcula, no se guarda	Nunca queda desactualizado: cambia apenas se carga una asistencia o una entrega	`RiesgoService` se llama al abrir cada pantalla

Decisión	Por qué	Dónde se ve
Una sola regla de riesgo	Si Inicio, Panel e Historial calcularan cada uno por su cuenta, un mismo alumno podría salir con dos colores	`RiesgoService.CalcularRiesgo`
Dos señales (asistencia y entregas)	La clienta dice "asistencia" pero sus casos empezaron por dejar de entregar	Panel e Historial muestran las dos
La justificada vale la mitad	Una falta avisada pesa menos que una sin explicación, pero no se borra	Constante en `RiesgoService`
Clase sin registro = ausente	Si no hay dato, la tutora tiene que enterarse	`RiesgoService` y el historial ("Sin registro")
Una clase por comisión y fecha, y un registro por alumno y clase	Guardar dos veces actualiza y no duplica	Restricciones únicas del script y recarga de filas tras guardar
La carga masiva muestra una vista previa y guarda todo o nada	Un DNI repetido no deja la importación a medias	`CargaMasivaViewModel.EvaluarAsync` y `SaveEstudiantesMassiveAsync`
Selector de comisión en lugar de texto libre	Evita comisiones duplicadas por un error de tipeo ("A" y "Comision A")	Pantallas de Alumnos y Carga masiva
`IDataService` como única interfaz	La persistencia es lo más probable que cambie; el resto se simplifica	`MauiProgram.cs`

Decisión	Por qué	Dónde se ve
`SeleccionAlumnoService` para abrir la ficha	Más simple y estable que pasar parámetros por la navegación entre pestañas	Panel e Historial
La base se arma con scripts, sin migraciones	El profesor o cualquiera puede recrearla en minutos en su equipo	Carpeta `scripts/`
Cuatro historias quedan para los sprints 6 y 7	Antes de la demo había que priorizar el ciclo completo con dos señales	Tablero de Trello

El diseño inicial era otro. Las tarjetas de Trello muestran un primer diseño con SQLite, un solo servicio de base de datos y un cálculo solo por inasistencias. El sistema actual usa SQL Server con Entity Framework Core, una interfaz de acceso a datos y dos señales. Conviene tener clara la razón de cada cambio antes de la defensa.

11. Limitaciones conocidas y cómo decirlas

Conviene nombrar las limitaciones vos antes de que las descubra el profesor, y presentarlas como decisiones conocidas con una mejora prevista, no como errores.

Limitación	Qué pasa	Cómo decirlo
Alumno nuevo en rojo	Un alumno recién cargado en una comisión con clases dadas aparece en rojo hasta que se cargue su asistencia	"Sin dato, el sistema alerta. Una mejora es exigir un mínimo de clases antes de evaluar"
Pocas clases	Con una sola clase, una ausencia da 0 % de asistencia y rojo	Lo mismo: el mínimo de clases y de trabajos es una mejora prevista
Justificadas	Valen la mitad y no se descartan	"Es una regla que acordamos; está en una constante de `RiesgoService` y se cambia en un solo lugar"

Limitación	Qué pasa	Cómo decirlo
Umbrales fijos	75/85 de asistencia y 60/80 de entregas están en el código	H7 los vuelve configurables; está planificada
Sin intervenciones	No se registra a quién se contactó ni cuándo	H10 y H11 están escritas y planificadas para los sprints 6 y 7
Calendario parcial	Las clases nacen al guardar asistencia y no hay una vista de calendario	H2 está planificada
Sin edición ni baja de alumnos	Solo alta y consulta	Quedó fuera del alcance en el análisis de requerimientos (RF08)
Comisiones por script	La app no crea comisiones	Se crean con `03_comisiones.sql`; es un dato maestro
CSV	Ignora filas con menos de tres columnas y exige UTF-8	La vista previa muestra el resto de los problemas
Email	Se valida el formato, no que la casilla exista	Alcanza para evitar errores de carga
Un solo usuario	No hay login	La consigna define un único usuario: quien sigue la comisión
Pruebas	Unitarias sí; de integración y de aceptación todavía no	El calendario las ubica en el sprint 8
Script de creación	`01_crear_base.sql` borra la base cada vez que se corre	Es para armarla de cero; el comportamiento se revisa después de la demo

Qué sí se puede afirmar con firmeza: el ciclo completo funciona de punta a punta con dos señales, y el cálculo tiene 25 pruebas unitarias que lo respaldan.

12. Guion de la demo

La demo cuenta una historia de unos 9 minutos: del problema a la respuesta "¿a quién llamo?", y cómo esa respuesta cambia en vivo. Usá la comisión A, que tiene los casos más claros.

Minuto	Pantalla	Qué hacer	Qué decir
0:00	Ninguna	Presentar el problema	"La tutora tiene la asistencia en un cuaderno y las entregas en el aula virtual. Cruzarlas a mano lleva una tarde, así que se entera tarde de quién dejó de venir"
1:00	Panel de riesgo, comisión A	Mostrar la lista ordenada	"Los rojos van primero. Cada alumno trae el dato que justifica el color: asistencia y entregas"
2:00	Ficha de Joaquín	Tocarlo en el panel	"Viene a todas las clases pero entregó 1 de 3 trabajos. Con solo asistencia saldría verde: por eso usamos dos señales"
3:00	Asistencia por fecha	Comisión A, fecha de hoy; marcar a Mateo ausente sin justificar y guardar	"Se toma la asistencia de toda la comisión en una pantalla. La misma sirve para corregir"
4:00	Panel	Volver: Mateo pasó a rojo	"El riesgo se recalcula solo, no se guarda"
4:30	Asistencia por fecha	Corregir a Mateo como ausente justificado y guardar; mostrar el panel (amarillo). Después corregir a presente (verde)	"Una falta justificada vale la mitad: de rojo a amarillo. Con presente, verde"
6:00	Entregas	Marcar la entrega de Joaquín en el TP2 y guardar; volver al panel	"Pasa de rojo a amarillo. Estos son los trabajos que la tutora carga"
7:00	Carga masiva	Comisión A, elegir `alumnos_demo.csv`, mostrar la vista previa e importar	"Antes de guardar muestra qué es nuevo, qué ya existe y qué tiene errores. Guarda todo o nada"

Minuto	Pantalla	Qué hacer	Qué decir
8:30	Cierre	Resumir	"Cada pantalla habilita una decisión. Quedan H2, H7, H10 y H11 para los sprints 6 y 7"

Los números de Mateo, para no improvisar. Con 7 clases (las 6 de la demo más la de hoy): ausente injustificado 5 de 7, 71 %, rojo. Ausente justificado 5,5 de 7, 79 %, amarillo. Presente 6 de 7, 86 %, verde.

Antes de empezar

- Corré los scripts `01`, `03` y `04` ese mismo día o el anterior.
- Dejá la app abierta en el Panel, con la comisión A elegida.
- Tené `datos_demo/alumnos_demo.csv` a mano.

Para ensayar sin recrear la base: volver a correr `04_datos_demo.sql` borra y recrea los datos de las comisiones A, B y C (incluida la clase de hoy y los alumnos importados) y deja todo como al principio.

Si algo falla en vivo: seguí con la siguiente pantalla y decí qué ibas a mostrar. Un error explicado a tiempo pesa menos que quedarse trabado.

13. Preguntas probables y respuestas

La pregunta que la consigna asegura que van a hacer es qué decisión habilita cada pantalla. Respondela con esta tabla y después seguí con el resto.

Pantalla	Decisión que habilita
Panel de riesgo	A quién contactar primero esta semana
Historial del alumno	Por qué ese alumno está en riesgo y qué decirle
Asistencia por fecha	Es la fuente de la señal de asistencia; permite corregir errores de carga
Entregas	Es la fuente de la señal de entregas; detecta a quien viene pero no entrega
Alumnos y carga masiva	Quién forma parte de cada comisión
Inicio	Si hay un problema grande antes de entrar al detalle

¿Cómo se calcula el riesgo? Con dos porcentajes: asistencia (las justificadas valen la mitad) y entregas. Rojo si la asistencia es menor al 75 % o las entregas menores al 60 %. Amarillo si son menores al 85 % o al 80 %. Verde en el resto. La regla está en `RiesgoService`.

¿Por qué dos señales y no solo asistencia? Porque la clienta dice que la señal es la asistencia, pero los casos que cuenta empezaron por dejar de entregar. Joaquín viene a todas las clases y entregó 1 de 3: solo con asistencia no se vería.

¿Qué pasa con un alumno que recién se cargó? Si su comisión ya tiene clases, aparece en rojo hasta que se cargue su asistencia, porque una clase sin registro cuenta como ausencia. Es una limitación conocida; un mínimo de clases la resolvería.

¿Por qué la justificada vale la mitad? Es una regla que definimos: una falta avisada pesa menos que una sin explicación, pero no desaparece. Está en una sola constante de `RiesgoService`, así que cambiarla es una línea.

¿Por qué MVVM? Separa lo que se ve de lo que se calcula. Los ViewModels no conocen el XAML ni la base de datos directamente, y eso permite probar la lógica sin abrir pantallas.

¿Por qué SQL Server con Entity Framework Core? Permite restricciones (claves foráneas y únicos) que protegen los datos, y se arma con un script. La interfaz `IDataService` aísla el motor: cambiarlo no tocaría los ViewModels. El diseño inicial usaba SQLite; confirmá con tu grupo el motivo exacto del cambio antes de la defensa.

¿Cómo evitan duplicados? En dos niveles. La pantalla avisa antes de guardar un DNI repetido en la comisión, y la base lo rechaza con una restricción única si algo se le escapa. Lo mismo con una asistencia o una entrega por alumno.

¿Qué pasa si falla la base? Los ViewModels atrapan el error, lo mandan a la ventana de salida y muestran un aviso o una lista vacía. La app no se cierra.

¿Cómo prueban? 25 pruebas unitarias xUnit sobre el cálculo y la validación del email, más una verificación manual de cada pantalla con los datos de demo. Las pruebas de integración y de aceptación están en el sprint 8.

¿Por qué no están H10 y H11? Están escritas en el tablero y planificadas para los sprints 6 y 7. Para el 16/10 desarrollamos 7 historias, una más de las 6 que exige el calendario, y priorizamos el ciclo completo con dos señales.

¿Cómo evolucionaría el sistema? Los umbrales (H7) se volverían configurables en `RiesgoService`, sin tocar las pantallas. Las intervenciones (H10 y H11) sumarían una entidad, métodos en `IDataService` y una pantalla. Cambiar de motor de base de datos solo requiere una nueva implementación de `IDataService`.

¿Cómo importan alumnos? Se elige comisión y archivo, la pantalla muestra una vista previa con el estado de cada fila, y recién entonces se importa. Valida campos obligatorios, email, DNI repetido en el archivo y DNI que la comisión ya tiene.

Si te piden ver el código de algo, abrí esto

Si piden ver	Archivo	Método o zona
La regla de riesgo	`Services/RiesgoService.cs`	`CalcularRiesgo`
Cómo se ordena el panel	`ViewModels/PanelRiesgoViewModel.cs`	`CargarAlumnosAsync`
Cómo se registra la asistencia	`ViewModels/AsistenciaFechaViewModel.cs`	`GuardarAsync`
Cómo se guardan las entregas	`ViewModels/EntregasViewModel.cs`	`GuardarAsync`
La importación de CSV	`ViewModels/CargaMasivaViewModel.cs`	`EvaluarAsync` e `ImportarAsync`
La validación del email	`Services/ValidacionEmail.cs`	`EsValido`
El acceso a la base	`Services/SqlServerDataService.cs`	Un método cualquiera
La conexión	`Services/AppDbContext.cs`	La variable `PPII_DB`
El registro de piezas	`MauiProgram.cs`	Toda la clase
Las pestañas	`AppShell.xaml`	Los `Tab`
El esquema de la base	`scripts/01_crear_base.sql`	Todo
Las pruebas	`ProyectoFinal-PPII-G6.Tests/`	`RiesgoServiceTests.cs`

14. Checklist del día de la demo y plan B

Tres cosas cierran la mayor parte del riesgo: la base armada ese mismo día, la variable de conexión revisada y un ensayo completo en la PC de la demo.

Código y repositorio

- [ ] Pull request de `DEMO_16-10` mezclado en `main`, con el respaldo `respaldo-main-antes-demo` creado antes
- [ ] Tag `demo-16-10` creado y subido
- [ ] Clon nuevo de `main` compilado, con las 25 pruebas aprobadas
- [ ] Código congelado desde el jueves

Base de datos

- [ ] Scripts `01`, `03` y `04` corridos el día de la demo o el anterior (el `02` se saltea con autenticación de Windows)
- [ ] La tabla de verificación del `04` muestra A con 5 alumnos, B con 4 y C con 3, y 6 clases y 3 trabajos en cada una
- [ ] No volver a correr `01_crear_base.sql` si hay datos que se quieren conservar

La PC de la demo

- [ ] SQL Server Express en ejecución
- [ ] La variable `PPII_DB` vacía en una PowerShell nueva, o apuntando adonde corresponde
- [ ] Proyecto de inicio `ProyectoFinal-PPII-G6` y destino Windows Machine
- [ ] App compilada y abierta una vez antes de presentar, porque el primer arranque tarda
- [ ] Recorrido completo ensayado con cronómetro
- [ ] `datos_demo/alumnos_demo.csv` accesible

Tablero de Trello

- [ ] Descripciones de los dos diagramas al día (SQL Server, entidades nuevas, `RiesgoService`)
- [ ] Textos viejos de los checklists (SQLite y nombres de clases) corregidos

Plan B

- [ ] Capturas o un video corto del recorrido funcionando
- [ ] Scripts a mano para recrear la base en minutos
- [ ] Servidor remoto como alternativa si hay red: la conexión se cambia con `PPII_DB`

Comandos útiles (PowerShell)

Comando	Para qué
`Get-Service 'MSSQL$SQLEXPRESS'`	Ver si SQL Server Express está corriendo
`echo $env:PPII_DB`	Ver qué conexión tiene definida la PC
`[Environment]::SetEnvironmentVariable("PPII_DB", $null, "User")`	Borrar la variable para usar la base local por defecto
`dotnet test`	Correr las pruebas desde la raíz del repositorio
`git tag demo-16-10` y `git push origin demo-16-10`	Marcar la versión de la demo